Geospatial intelligence for environmental decisions

GEOINSIGHT

KAMRUP DISTRICT, ASSAM

Project report and technical documentation

A full-stack prototype combining a React dashboard, FastAPI, PostgreSQL/PostGIS-ready storage, public weather data, an ML baseline, and an interactive satellite map.

1. Executive Summary

GeoInsight turns difficult geospatial processing into a simple district-level environmental report. A user selects an Assam district and observation month; the application returns vegetation health, rainfall, surface-water coverage, a historical rainfall view, current weather, and baseline risk signals.

The prototype currently supports Kamrup, Kamrup Metropolitan, and Dibrugarh. The interface is designed for real Sentinel-2, CHIRPS, JRC Global Surface Water, and geoBoundaries data. Its API and database structure are ready for those datasets, while the current metric training records are explicitly labeled as demo data.

Problem statement

> *Public Earth observation datasets are valuable but difficult for normal applications to consume because they have different formats, resolutions, coordinate systems, and processing requirements. GeoInsight provides one standardized interface for turning those datasets into understandable environmental indicators.*

2. Goals and Outcomes

District selection	State and district selectors with three Assam districts.
Vegetation analysis	NDVI metric and vegetation layer slot in the map workflow.
Rainfall analysis	Monthly rainfall prediction, historical rainfall chart, and live weather card.
Surface-water analysis	Coverage percentage and calculated water area in square kilometres.
Disaster and field outlook	Baseline flood, drought, and crop-stress signals.
Application integration	FastAPI REST endpoints consumed by the React dashboard.

3. Architecture

The system is split into a browser dashboard, an API and processing layer, and a PostgreSQL persistence layer. The map uses Leaflet with Esri World Imagery and OpenStreetMap labels. Current weather is retrieved from Open-Meteo without an API key.

Request flow

```
User selection -> React dashboard -> FastAPI endpoint -> model/data services -> PostgreSQL records -> JSON response -> cards, charts, risks, and map layers
```

Technology stack

Frontend	React + TypeScript + Vite	Responsive dashboard, selectors, charts, map controls.
Map	Leaflet + Esri World Imagery	Interactive satellite map with district-aware center and overlays.
Backend	Python + FastAPI	REST API, validation, live weather retrieval, risk logic.
ML	scikit-learn Random Forest	Baseline next-month rainfall, NDVI, and water prediction.
Database	PostgreSQL, PostGIS-ready	Districts, environmental metrics, prediction runs, model runs.
External data	Open-Meteo, Esri, OpenStreetMap	Current weather and map tiles.

4. Processing Workflow

1. Input	Select Assam state, district, and month.
2. Boundary	Resolve the selected district boundary and area.
3. Data loading	Load satellite, rainfall, and surface-water datasets.
4. Spatial processing	Clip and mask records to the district boundary.
5. Indicators	Calculate NDVI, rainfall statistics, water percentage, and area.
6. Prediction	Train on historical records and test on unseen later records.
7. Delivery	Store results and expose standardized JSON through FastAPI.

5. API Documentation

GET /api/v1/health	Check API availability.
GET /api/v1/districts	Return supported districts.
GET /api/v1/prediction?district=Kamrup&month;=2026-07	Return prediction, risks, water area, and rainfall history.
GET /api/v1/live?district=Kamrup	Return current Open-Meteo conditions.
GET /api/v1/model/validation	Return MAE, RMSE, R2, test period, and database status.

Example prediction response


```
{"district":"Kamrup","state":"Assam","prediction":{"rainfall_mm":400.8,"ndvi":0.599,"water_percent":4.97,"water_area_km2":215.9},"risks":{"flood_risk":"high","drought_risk":"low","crop_stress":"low"}}
```


6. Machine Learning and Validation

The current model is a simple RandomForestRegressor baseline. It uses chronological validation: earlier records are used for training and the latest records are held out for testing. The final model then trains on the full available history for future prediction.

Rainfall (mm)	54.2	62.2097	0.7483
NDVI	0.0188	0.0197	-0.3472
Surface water (%)	0.267	0.2882	0.657

These metrics are from the built-in demo dataset and are not official accuracy claims. The report must be updated after government-validated historical data is loaded.

7. Database Design

districts	District name, state, area, and boundary GeoJSON.
environmental_metrics	Monthly NDVI, rainfall, water coverage, area, and source.
prediction_runs	Predicted indicators, target month, model, and training source.
model_training_runs	Train/test counts, test period, MAE, RMSE, R2, and model metadata.

8. Testing and Verification

Frontend TypeScript/Vite production build	Passed
Python backend compilation	Passed
Prediction endpoint	HTTP 200
District list endpoint	Three Assam districts returned
Live weather endpoint	HTTP 200 when Open-Meteo is reachable
Historical rainfall response	18 records returned from demo history
PostgreSQL schema	Tables created and district seed verified
Model validation persistence	Validation run stored in PostgreSQL

9. Setup and Demonstration

Prerequisites: PostgreSQL running locally, Python with backend dependencies, and Node.js/npm. From the project root, run the one-command launcher:

```
run.bat
```

Dashboard: http://127.0.0.1:5173/ | API docs: http://127.0.0.1:8001/docs

Select a district and month, toggle map layers, inspect historical rainfall, open processing details, and review field/disaster outlook cards. The API source field identifies demo data until real government datasets are loaded.

10. Limitations and Next Steps

The current environmental history is a clearly labeled demo dataset. District profile adjustments make the multi-district interface demonstrable, but they are not a replacement for actual district raster/vector processing. The next production step is to load geoBoundaries, Sentinel-2, CHIRPS, and JRC data, clip them with PostGIS/rasterio, store derived metrics, and retrain/evaluate the model on real historical observations.

> *Project status: working prototype, API and database verified, real government geospatial ingestion pending.*